

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Y. Ramadevi	Reg. No.:	192373065
Date:	21/08/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Real-Time Application Monitoring and Automated DevOps Maintenance Using Prometheus, Grafana, and Terraform

1. Introduction

The proposed system, “Real-Time Application Monitoring and Automated DevOps Maintenance Using Prometheus, Grafana, and Terraform,” is designed to continuously observe application and infrastructure health while automating infrastructure provisioning and maintenance. The CI/CD pipeline ensures that monitoring configurations, dashboard definitions, infrastructure code, alert rules, and application changes are validated and delivered in a controlled manner.

The pipeline combines Git-based source control, automated build and testing, infrastructure validation with Terraform, container/image quality checks, deployment to development and staging environments, controlled production release, and continuous monitoring through Prometheus and Grafana. Automated alerts and rollback procedures reduce downtime and improve operational reliability.

2. Project Requirements and CI/CD Needs

2.1 Functional Requirements

Collect application and infrastructure metrics such as CPU, memory, request rate, latency, error rate, disk usage, and service availability.

Expose application metrics in a Prometheus-compatible format and collect them at configured intervals.

Display real-time and historical operational information through Grafana dashboards.

Define alert rules for high resource utilization, service failure, high latency, and abnormal error rates.

Provision and update infrastructure using Terraform instead of manual configuration.

Automatically validate monitoring configuration, Terraform code, application code, and deployment manifests.

Deploy tested changes through development, staging, and production environments.

Support rollback to the last known-good release when deployment or health checks fail.

2.2 Non-Functional Requirements

High availability and fast detection of service failures.

Repeatable and auditable infrastructure provisioning.

Secure handling of credentials, API tokens, and deployment secrets.

Fast feedback after every source-code or infrastructure-code change.

Traceability from commit to build, test result, deployment, and monitoring status.

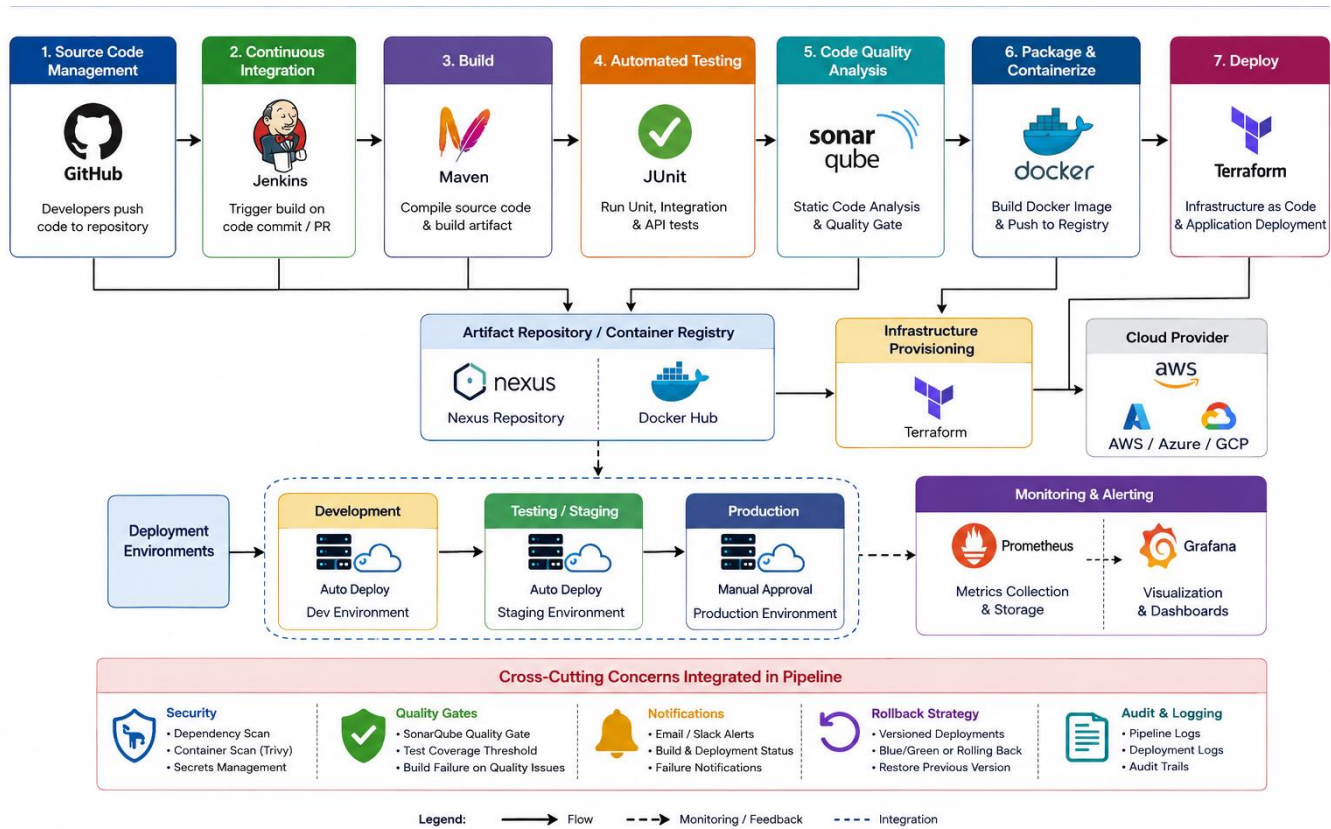
Scalability for increasing application traffic and monitored resources.

2.3 CI/CD Requirements

Requirement	CI/CD Need
Source control	Git repository with protected main/production branch and pull-request review.
Continuous integration	Automatic pipeline execution on pull requests and approved commits.
Build/package	Create versioned application/container artifacts.
Testing	Run unit, integration, configuration, and infrastructure validation tests.
Quality/security	Static analysis, dependency scanning, secret scanning, and image scanning.
Infrastructure	Terraform fmt, validate, plan, policy checks, and controlled apply.
Deployment	Separate development, staging, and production environments.
Monitoring	Prometheus health metrics and Grafana dashboards after deployment.
Recovery	Health checks, deployment gates, automatic/manual rollback.

3. Proposed CI/CD Pipeline Architecture

The following workflow is proposed for the system:



4. Pipeline Stages and Workflow Explanation

Stage	Workflow
1. Source Code Management	Developer pushes code, Terraform files, Prometheus rules, Grafana dashboard definitions, and deployment manifests to Git. Pull requests trigger validation before merging.
2. Checkout	The CI runner checks out the exact commit so every pipeline execution is reproducible and traceable.
3. Code Quality	Linting and static analysis identify syntax errors, maintainability problems, duplicated code, and common defects.
4. Automated Tests	Unit tests verify application functions; integration tests verify communication among application, metrics endpoint, and supporting services.
5. Security Checks	Dependency scanning, secret scanning, static security checks, and container-image vulnerability scanning are performed before deployment.

6. Terraform Validation	Terraform formatting and validation are executed. Terraform plan previews infrastructure changes; policy checks can prevent unsafe configurations.
7. Build and Packaging	A versioned application artifact or Docker image is created and stored in an artifact/container registry.
8. Development Deployment	The validated artifact is deployed automatically to the development environment. Smoke tests verify basic availability.
9. Staging Deployment	Staging receives the same immutable artifact. Integration, end-to-end, performance, and infrastructure tests are executed.
10. Production Gate	A quality gate checks test results, security status, Terraform plan, and application health. Production deployment may require approval.
11. Production Deployment	The approved version is deployed using a controlled strategy such as rolling or blue-green deployment.
12. Monitoring and Feedback	Prometheus collects metrics and Grafana presents dashboards. Alerting identifies abnormal behavior. Failed health checks initiate rollback or stop further rollout.

5. Tools and Technology Selection

Pipeline Area	Selected Tool / Technology	Justification
Source Control	Git + GitHub/GitLab	Version control, branching, pull requests, review, and audit history.
CI/CD Automation	GitHub Actions / GitLab CI	Automates repeatable build, test, security, Terraform, and deployment workflows.
Application Packaging	Docker	Provides consistent runtime packaging across environments.
Infrastructure as Code	Terraform	Declarative, repeatable provisioning and controlled infrastructure changes.
Metrics Collection	Prometheus	Time-series metrics collection and rule-based monitoring.
Visualization	Grafana	Dashboards for application, infrastructure, and CI/CD health.
Testing	JUnit/PyTest/npm test or equivalent	Automated unit and integration validation according to application stack.
Static Analysis	SonarQube / equivalent	Identifies code-quality and maintainability issues.
Security	Trivy + secret/dependency scanner	Scans container images and dependencies for known

		vulnerabilities and accidental secrets.
Artifact Storage	Container Registry / Artifact Registry	Stores immutable versioned build artifacts.
Notifications	Email / Slack / Teams	Provides pipeline and production-alert notifications.

Tool selection can be adapted to the laboratory environment. The important requirement is that every selected tool supports automation, repeatability, traceability, and integration with the CI/CD pipeline.

6. Automated Testing Strategy

Test Type	When Executed	Purpose
Unit Testing	Every pull request / commit	Validate individual application functions and modules.
Integration Testing	After successful build	Verify application, database/API dependencies, metrics endpoint, and service interactions.
Terraform Validation	Every infrastructure change	Detect invalid Terraform configuration before infrastructure changes are applied.
Terraform Plan Review	Pull request / deployment	Show intended infrastructure changes and prevent unexpected modifications.
Static Code Analysis	Every CI run	Detect code-quality and maintainability issues.
Dependency Scan	Every CI run	Identify vulnerable third-party packages.
Secret Scan	Every CI run	Prevent credentials and sensitive tokens from entering the repository.
Container Scan	After image build	Detect known vulnerabilities in image packages and dependencies.
Smoke Testing	After each environment deployment	Confirm service availability and key endpoints.
End-to-End Testing	Staging	Validate important user/system workflows across components.
Performance Testing	Staging / scheduled pipeline	Evaluate response time, throughput, and resource behavior under load.
Monitoring Validation	Post-deployment	Confirm Prometheus targets, metrics, dashboards, and alert rules work correctly.

7. Deployment Environment Strategy

Environment	Purpose	Deployment Method	Gate
Development	Developer/integration validation	Automatic after CI success	Automated tests
Staging	Production-like verification	Automatic or controlled promotion	Integration, E2E, security and performance checks
Production	Live service	Controlled rolling/blue-green deployment	Approval + all quality gates + health checks

The same versioned artifact should be promoted from development to staging and then production rather than rebuilding a different artifact for production. This reduces the risk of environment-specific build differences.

8. Terraform-Based DevOps Automation

Terraform configuration is stored in version control along with environment-specific variables and modules where appropriate.

CI runs terraform fmt and terraform validate on every infrastructure change.

terraform plan is generated for review before changes are applied.

Sensitive values are supplied through secure CI/CD secrets or a dedicated secret-management system rather than committed to Git.

Production terraform apply should require an explicit approval or protected environment.

Terraform state should be stored remotely with locking and access control when supported by the chosen backend.

After infrastructure changes, automated health checks confirm that monitored services and exporters are reachable.

9. Prometheus and Grafana Integration

Prometheus acts as the metrics collection and alert-rule evaluation layer. The application exposes metrics, while infrastructure exporters can provide host and system metrics. Grafana connects to Prometheus as a data source and presents dashboards for operational visibility.

Monitoring Area	Example Metric / Indicator	Action
Availability	Service up/down status	Alert operations team when service is unavailable.
CPU	CPU utilization	Alert when sustained utilization crosses threshold.
Memory	Memory utilization	Investigate leaks or resource pressure.
Latency	Request response time	Trigger alert for sustained high latency.

Errors	HTTP 5xx/error rate	Identify application or dependency failures.
Capacity	Disk usage / resource saturation	Scale or clean resources before outage.
CI/CD	Build/deployment success rate	Detect pipeline reliability problems.
Infrastructure	Terraform-managed resource health	Verify infrastructure after changes.

10. Security and Quality Gates

Use branch protection and mandatory pull-request review for production-bound changes.

Store passwords, tokens, cloud credentials, and webhook secrets in protected CI/CD secret storage.

Apply least-privilege permissions to CI runners, Terraform, registries, and deployment accounts.

Scan source dependencies and container images for known vulnerabilities.

Block the pipeline when critical security findings or failed mandatory tests are detected.

Do not print secrets in CI logs. Mask sensitive variables and restrict log access.

Use signed/versioned artifacts where supported and retain build metadata for traceability.

Keep Terraform state and production configuration protected from unauthorized modification.

11. Deployment and Rollback Strategy

A rolling deployment is suitable for the proposed system because it gradually replaces instances while maintaining service availability. For higher-risk releases, a blue-green strategy can be used so traffic can switch back to the previous environment quickly.

Condition	Pipeline Action
All tests pass and health checks succeed	Continue deployment.
Smoke test fails in development	Stop promotion and report failure.
Staging quality gate fails	Block production deployment.
Production health check fails	Stop rollout and initiate rollback.
High error/latency alert after release	Evaluate release health and rollback if threshold persists.
Rollback succeeds	Restore previous stable version and notify stakeholders.
Rollback fails	Escalate to operations team and keep affected deployment isolated where possible.

Rollback should use the last known-good immutable artifact and, where infrastructure changes are involved, a reviewed infrastructure rollback plan. Database changes should be designed to be backward compatible whenever possible.

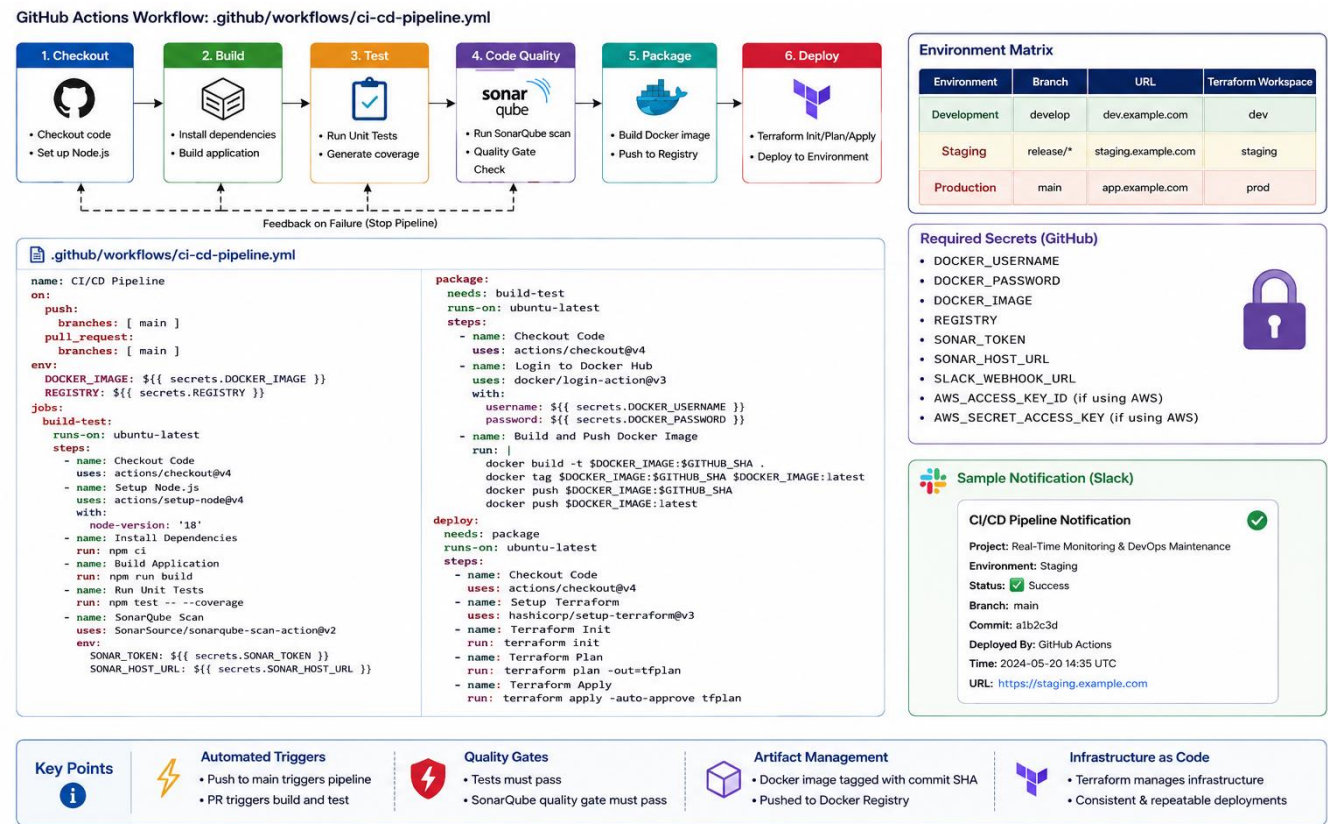
12. Notifications and Feedback Mechanism

- Notify the development team when CI fails.
- Notify reviewers when a pull request is blocked by quality or security gates.
- Notify DevOps/operations when staging or production deployment starts and finishes.
- Send immediate alerts for critical Prometheus conditions.
- Notify stakeholders when rollback occurs.

Include commit ID, pipeline stage, environment, failure reason, and dashboard/log references in notifications.

13. Example CI/CD Workflow Configuration

The following pseudocode illustrates the intended automation sequence. The exact syntax depends on the selected CI platform.



14. End-to-End Workflow Explanation

Developer creates or modifies application, Terraform, monitoring, dashboard, or deployment configuration.

Changes are committed to Git and submitted through a pull request.

CI automatically checks formatting, code quality, tests, dependencies, secrets, and security.

Terraform is validated and a plan is generated for infrastructure changes.

After successful checks, the application is built and packaged as a versioned artifact/container image.

The image is scanned and stored in a registry.

The artifact is deployed to development and smoke-tested.

The same artifact is promoted to staging for integration, end-to-end, and performance validation.

After all quality gates and required approval, production deployment begins.

Prometheus collects live metrics and Grafana displays operational dashboards.

If health checks and monitoring remain healthy, deployment is marked successful.

If serious failures occur, rollout is stopped and the previous stable release is restored.

15. Expected Benefits

Faster and safer software delivery through automation.

Reduced manual infrastructure configuration through Terraform.

Early detection of application and infrastructure failures.

Improved visibility using Prometheus metrics and Grafana dashboards.

Consistent deployment across environments.

Reduced deployment risk through automated testing and quality gates.

Faster recovery using a defined rollback process.

Better auditability because code, infrastructure, artifacts, and deployment history are version controlled.

16. Conclusion

The proposed CI/CD design integrates continuous integration, infrastructure as code, automated testing, security validation, controlled deployment, and continuous monitoring for the real-time application monitoring system. Git provides traceability, CI automation provides repeatable validation, Terraform automates infrastructure maintenance, Docker provides consistent packaging, and Prometheus with Grafana provides continuous operational feedback. Together, these components create a reliable DevOps workflow that can detect failures early, prevent unsafe releases, and recover quickly through rollback.